

QComputations: Библиотека для квантовых вычислений

Андрей Кузьминский

*Московский Государственный Университет им. Ломоносова,
Факультет Вычислительной Математики и Кибернетики, Воробьёвы Горы 1, Москва, 119991, Россия
(12 сентября 2024 г.)*

Реализована новая библиотека для квантовых вычислений. С её помощью намного проще решаются задачи моделирования любых квантовых систем и любых областей, будь то квантовая химия, криптография, компьютер, переборные задачи и так далее. Также библиотека сравнивается с другими уже известными, но преимущественно - Qutip, так как его цели совпадают с моей.

Keywords: ПО, Библиотека, C++, Intel OneAPI, CUDA, Распределённые вычисления, Параллельные вычисления, Квантовые вычисления, Квантовая механика, Квантовая криптография, Квантовая химия, Биология

I. Введение

Квантовая механика совершила революцию во многих областях науки: в физике, в химии, в программировании и так далее. В программирование же квантовая механика породила ответвление, которое сейчас называется квантовыми вычислениями. Данная область охватывает многие задачи: начиная от моделирования самой квантовой (а может и классической) физики, заканчивая той же химией, биологией, криптографией, методами оптимизации и др. Если же обобщать - данная сфера занимается моделированием квантовых экспериментов, которые же, в свою очередь, могут применяться к чему-то конкретному. Одной из глобальных задач же является создание квантового компьютера - машины, способной работать с квантовыми частицами (или с системами частиц) напрямую и использовать их (а точнее квантовые эффекты) для решения задач. На пути к её решению у учёных появилось не мало проблем. Самая известная из них - декогеренция, спонтанное измерение системы частиц со стороны природы. Ею занимаются и наша область, но есть проблема куда более приземлённая, но не менее важная.

Как было сказано квантовые вычисления занимаются моделированием квантовой физики. Её математический аппарат в некотором смысле уникален: в нём абсолютно всё взаимосвязано, то есть нет какой-то основополагающей абстракции, которая бы являлась основой всему остальному, как например понятие группы в теории групп, колец и полей. Но несмотря на это, этот аппарат всё-таки придерживается строгих правил и законов. Проблема звучит несерьёзно, но беда кроется в одной ремарке. Дело в том, что, как и в задаче предсказания погоды, решения не устойчивы к условиям задачи, то есть решение слишком сильно меняется от минимального изменения условия. В квантовых вычислениях проблема глубже: решение может поменяться не численно, а качественно. В одних задачах удобно начинать решение с понятия операторов, другие - с состояний, в одних дискретизировать непрерывный оператор в обычную матрицу, в другой

- работать с оператором как с разреженной, или вообще с функцией, одна решается быстрой явной схемой, а при добавлении хотя бы одного оператора декогеренции - только медленными итеративными методами, которые заново нужно параллелить, отлаживать и так далее.

Если резюмировать, была проблема в создании универсального программного инструментария для квантовых вычислений, а именно такого, чтобы на нём решить задачу было проще (и возможно!), чем реализовывать самому с нуля. Звучит слишком абстрактно, поэтому я приведу 3 главные задачи, которые должен решать программный пакет для квантовых вычислений.

1. Квантовые вычисления занимаются физическими задачами даже в программных вопросах, поэтому в пакете должно быть удобно и понятно работать в первую очередь в нотациях квантовой механики, то есть в нотациях математических формул и операторов. Реализация этих операторов также должна быть эффективной и удобной. Это частично реализовано в Qutip, правда он поддерживает только матричный вид данных (разреженные матрицы большой погоды также не делает), так как у него жёсткая привязка к классам QObj. Подробнее в документации к Qutip.
2. С экспоненциальным ростом от числа кубитов не только сложности вычислений, но и во многих задачах ещё и памяти, параллелизация является не бонусом, а необходимостью. Также желательно, чтобы библиотека поддерживала и разнообразную параллелизацию, написанную над самой библиотекой. Это сильно страдает в Qutip даже на нитях, суперкомпьютерной же версии у него нет в принципе, без которой многие задачи становятся невозможно решить.
3. Пакет должен упрощать жизнь программисту. Большинство библиотек в интернете по квантовым вычислениям либо решает узкий круг задач, из-за чего их применение на другие

становится принципиально невозможным, либо крайне неудобен. Например, Q# Майкрософта, который можно использовать только для строгого с физической точки зрения моделирования задач на кубитовом квантовом компьютере, что является серьёзным ограничением, так как некоторые модели становится невозможно реализовать. Например, модель Тависа-Каммингса-Хаббарда, которая требует в кубитах, отвечающих за фотоны, значения состояний больше 1.

На момент написания данной статьи лишь 1 популярная библиотека имеет схожие цели с представленной здесь - Qutip. Далее с ним будут сравнения как в эффективности, так и качественные: в гибкости, в возможностях, в удобстве и в др.

В разделе 2 будет введена краткая теория, на основе которой далее будет объясняться структура библиотеки и примеры. В 2 - структура самой библиотеки QComputations с некоторыми базовыми примерами по каждому этапу. Примеры и объяснения будут на одноядерной версии. В 3 - полноценные примеры для реализации Тависа-Каммингса, реализация алгоритма Гровера, эффект квантового туннелирования, а также нетипичное для квантовых вычислений - работа, основанная на необратимых операторах. В 4 сравнение эффективности, а также разбор параллельной версии и её взаимодействие с одноядерной. В 5 сравнение с библиотекой Qutip. В приложении А дана небольшая выдержка по основам квантовой теории и методах, которые будут использоваться в примерах из раздела 3. В Приложении Б будут тексты исходных программ для каждого примера. В Приложении В приведён код для объяснения работы самой библиотеки. В Приложении Г вкратце рассказано о том, что используется в библиотеке из Intel OneAPI и некоторые примеры реализации.

II. QCOMPUTATIONS

Когда я только начинал писать данную библиотеку, моя первая же идея была точно такая же, какая и у авторов Qutip - взять строгий математический аппарат в квантовой механике, а именно Дираковскую нотацию, и дальше на уровне кубитов, тензорных произведений и матриц. Данная версия хоть и обладала хорошей структурированностью, но не отличалась безумной эффективностью, удобством в задании произвольных систем и операторов. Было позже решено начинать отталкиваться от понятия гамильтонианов. Данный способ уже был лучше, но не покрывал большинство задач, а также всё равно был неудобен в задании произвольных систем. После чего библиотека приобрела тот вид, что она имеет сейчас.

Библиотека QComputations реализована на C++ и на пакете с открытым исходным кодом - Intel OneAPI, благодаря которому удалось добиться хоро-

шой эффективности. Разработка 99% всех программ для квантовых вычислений делится на 4 этапа:

1. Описание структуры состояния/системы. Состояния и системы бывают разные: разные кудиты (многозначные кубиты), разное разбиение на группы, разный вид и смысл и так далее
2. Задание операторов. Здесь будет представлено одно из главных преимуществ библиотеки.
3. Основная работа программы: инициализация состояний, гамильтонианов, операторов, систем, работа с ними, моделирование и так далее.
4. Запись результатов и визуализация.

А. Описание структуры состояний/систем

Структура библиотеки начинается с понятие состояния, и здесь библиотека уже перестаёт быть типичной в том плане, что, если абсолютное большинство библиотек предлагают готовые решения некоторых задач, методов или какого бы то ни было ещё функционала, то здесь на простое программирование накладывается некоторые ограничения, при условии которых, библиотека будет работать со всем, что Вы написали.

В библиотеке реализовано понятие (класс) - Basis_State (B.1). Данный класс является основополагающим во всей библиотеке и представляет собой наше базисное состояние. Как видно из Код. ??, он представляет собой 3 массива - значение самих кудитов (qudits_), их максимальные значения (max_vals_, хранятся, только если максимальное значение не равно 1), и группы (groups_) в целях удобства индексации.

Примечательно здесь же то, что всё, что вам нужно, чтобы реализовать своё понятие состояния - это просто объявить ваш новый класс, как дочерний от Basis_State. Это всё! Дальше Вы реализовываете ваше понятие состояния как хотите - с любыми дополнительными данными, параметрами, методами, функциями и так далее. Всё это в дальнейшем будет работать и будет совместимо со всей остальной библиотекой. Простой пример реализация Тависа-Каммингса приведён в Код. ?? В 3 разделе также будет приведён пример с дискретизацией непрерывного состояния.

Но, всё это является лишь базисным понятием состояния, суперпозиции здесь никакой нет. Для неё в библиотеке реализован специальный шаблонный класс - State<StateType> (B.2). Он хранит в себе комплексный вектор, а также базис, над которым он работает. В Код. ?? приведены примеры инициализации и модификации этого класса. Примечательно в нём в том, что, в отличие от Qutip, вам нужно инициализировать в нём лишь состояния с ненулевыми амплитудами (Приложение А), библиотека его потом сама расширит.

В. Задание операторов

Как уже было сказано выше, одной из версий было задание операторов в матричном виде. Это оказалось не только неэффективно, но и, как выясняется, и не нужно. Квантовая механика так устроена, что действие абсолютного большинства операторов можно описать абстрактно. Например, один оператор просто умножает состояние на сумму значений кубитов, другой - переводит значение 1 кубита в другое, третий - является суммой операторов, отвечающий за релаксацию и возбуждение атомов. Этот список бесконечен.

Для достижения этого, в библиотеке эксплуатируется понятие функции, а именно, её следующий прототип - `State<StateType> func_op(const StateType& state)`. То есть, на вход подаётся базисное состояние, а на выход - их суперпозиция. Данное решение позволяет не только реализовать любые действия над базисными состояниями и описать действие любого оператора, но и сделать его реализацию в разы эффективней, чем его экспоненциально растущее матричное представление.

После реализации всех операторов в виде функций, начинается главное преимущество `QComputations`. В ней реализован специальный класс `Operator<StateType>`, который позволяет объединять эти операторы в строгие математические формулы следующего вида:

Вы вольны реализовывать как и строгие формулы, так и их эквивалентные оптимизированные варианты. В репозитории самой библиотеки [?] в инструкции (`doc/instruction.pdf`) данное преимущество продемонстрировано во всей красе. Вы также можете при желании привести ваш оператор в матричный вид над каким-нибудь базисом (В.3).

Также, после такого как Вы реализовали сам оператор, Вы уже можете его использовать в своих вычислениях, а именно, прогонять через него ваши суперпозиции базисных состояний. Прототип этого метода выглядит так:

С. Основная работа программы

Мы описали с Вами систему и операторы над ней. Осталось только всё проинициализировать и приступить к моделированию.

Начнём с инициализации понятия состояния и суперпозиции. Поскольку мы уже реализовали наш собственный класс, то, логично, что у него уже есть конструктор. В Код. ?? приведены примеры инициализаций состояний вместе с выводами в комментариях. Вы вольны и инициализировать и сами комплексный вектор и базис, а потом просто передать в специальные методы, которые можно найти в (В.3).

После инициализации состояний, а также операторов, как это указано в предыдущем подразделе, следующим этапом может стать гене-

рация гамильтониана H (Приложение А). В библиотеке реализовано 2 основных способа его генерации - `H_by_Operator<StateType>` и `H_by_Scalar_Product<StateType>`. Прототипы обоих в (В.4). Первый, как легко догадаться, инициализирует H по оператору. Второй - по скалярному произведению вида $\langle i|H|j\rangle$, где $|i\rangle$ и $|j\rangle$ - это какие-то базисные состояния.

Заметьте, что мы до сих пор ни слова не сказали о том, как генерировать базис. Здесь фишка в том, что Вам в огромном числе задач и необязательно об этом думать, базис генерируется автоматически при инициализации гамильтониана с помощью алгоритма отбора рабочей области [?]. Его реализация лежит в файле `graph.hpp` в репозитории [?] (`src/QComputations/graph.hpp`). В него подаётся начальное состояние и оператор (также можно подавать и операторы декогеренции), и уже по ним генерируется базис (добавлять новые состояния до тех пор, пока они появляются, за этим и нужны максимальные значения кудитов). Если же Вы хотите проинициализировать таким способом базис сами - пример этого можно увидеть в (В.5).

После генерации гамильтониана остаётся только моделирование его действия над нашей системой, согласное различным уравнениям. В библиотеке на момент написания данной статьи (полный же список можно увидеть в документации к `QComputations`) реализовано эволюция уравнением Шредингера через спектральное разложение, а также с помощью прямого решения с подсчётом экспонент с помощью ряда Тейлора и Квантовое Основное Уравнение (КОУ). Этих методов достаточно для решения большинства задач в области квантовой химии и биологии, прогон состояний через операторы из предыдущего подраздела добавляет к охвату задач ещё большую часть: криптография, моделирование квантовых программ и компьютера. Дальше остаются лишь специфичные уравнения, которые, в принципе, можно реализовать по образу и подобию реализованных двух, так как исходный код полностью открытый, а в них самих же используется только доступный пользователю интерфейс (только публичные методы). Прототипы всех этих функций описаны в (В.6). Примеры с большей части из них будут приведены в разделе 3 и, соответственно, в приложении Б.

Д. Запись и визуализация

На данный момент (но не надолго) это самая слабая часть данной библиотеки в глобальном плане, но не в сравнительном, но об всём по порядку.

В библиотеке реализовано 2 способа визуализации: один с помощью Python API `Matplotlib` [?] (от которого я стремлюсь отказаться), и с помощью файловой системы, то есть через запись в файлы, которые, впоследствии, передаются вспомогательному скрипту,

использующему пакет Seaborn.

Начнём с Python API Matplotlib [?]. Данный способ удобен при условии, если Вам нужен результат сразу же после выполнения всех вычислений и, если Вы привыкли к Matplotlib. Поддерживаются также и 3D графики. Из плюсов - это всё, минусов у данного метода больше. Во-первых, чтобы изменить этот график, нужно лезть обратно в исходный код и его поправлять. По той же причине сильно затруднена кастомизация графиков. Если обобщать, в вопросе визуализации вместо самой визуализации и кастомизации нам приходится заниматься правками ради этого в исходном коде, что крайне неудобно. Во-вторых, Matplotlib не поддерживает ни в каком виде параллелизацию при генерации графиков хотя бы на нитях (Только независимых, что является по большей части исключением, и в разделе 4 приведён пример подобного). Из-за этого, запись и визуализация результатов становится узким местом в наших вычислениях, что немного фрустрирует. Поэтому, от данного метода я рекомендую отказаться в пользу временного второго на пакете Seaborn, так как в нём больше свободы, не нужно лезть обратно в исходный код ради кастомизации графика и поддерживается параллельная генерация gif изображений. Под конец подмечу, что Matplotlib - единственный способ визуализации в Qutip.

Второй способ - использование файловой системы и вспомогательного скрипта, использующего пакет Seaborn. После установки библиотеки у Вас появятся 2 новые переменные окружения: \$SEABORN_CONFIG - json файл с настройками для кастомизации графиков, и \$SEABORN_PLOT сам скрипт. Полная инструкция описана в инструкции в самом репозитории [?]. В библиотеке реализованы специальные функции для записи результатов в набор файлов. Базовый пример приведён в Код. ???. После этого Вы переходите в директорию с директорией результатов, как это показано на Рис. ??, настраиваете json файл, и запускаете скрипт.

Оба способа неидеальны, поэтому в будущем им на замену придёт специальное графическое приложение.

Е. QConfig

Как уже обсуждалось во введении, для разных задач подходят разные методы решения, в том числе и численные. Например, для решения КОУ, которое является матричным дифференциальным уравнением, для одних систем будет требовать точно метода, например Рунге-Кутты 4 порядка, а может и хватит куда менее точного, но более быстрого метода, как например, метода Эйлера. Но плодить несколько функций для разных способов решений идея сама по себе плохая, так как смысл функций не изменится. Для решения этой и подобной ей проблем был придуман специальный синглтон (класс, хранящийся в памяти в

единственном экземпляре), который настраивает глобальное поведение библиотеки. В нём можно выбирать как и алгоритмы для разных методов моделирования, так и точность чисел при записи в файл, разделители для универсального строкового представления базисных состояний, или, например, физических констант (постоянной планка, частоты и так далее). В Код. ?? приведён пример того, как это работает. Полный список параметров можно прочесть в документации в репозитории [?].

III. Примеры программ и моделей

В данном разделе рассматриваются примеры программ и моделей, реализованные на данной библиотеке. Здесь будут представлены примечательные куски кода, на которые хотелось бы обратить внимание. Для каждого примера также можно посмотреть в приложении Б основные части этих программ, кроме кода для необратимых операторов (для него код лежит в репозитории [?] для работы [?]). Полный код программ лежит в примерах в репозитории [?] в директории examples. Название всех программ для каждого примера соответственно указаны в приложении Б.

А. Тавис-Каммингс

Мы начнём с просто примера из квантовой химии - модели Тависа-Каммингса. Его краткая теория описана в приложении А. В нашей системе будет $m + 1$ кубит, где m - число атомов. Пусть n - число фотонов в нашей системе. Для упрощения частоты везде будут одинаковыми. Реализация модели с данным описанием продемонстрирована в Код. ???.

Теперь нам нужно реализовать гамильтониан. Реализацию всех операторов можно посмотреть в (Б.1), здесь же рассмотрим одно ключевое преимущество QComputations. Если в большинстве библиотек нужно скрупулёзно приводить каждый оператор к матричному виду, потом считать тензорные произведения, то в данной библиотеке действия операторов можно описать лишь абстрактно в виде действий над состояниями, но это не всё. Мы по-прежнему также можем реализовать каждый оператор по строгой математической формуле, как например возбуждение i -ого атома (реализация в Код. ???), но это как раз будет самый неэффективный способ. Сумма операторов по-прежнему является оператором, поэтому мы можем описать с помощью функций действия не одного оператора, а сразу целой суммы или произведения. Возьмём в качестве примера следующее слагаемое гамильтониана Это подсчёт энергии от атомов, которая включает в себя суммирование n тензорных произведений и n суммирования матриц. Мы же можем реализовать просто реализовать функцию, суммирующую энергию каждого атома (Код. ???).

Допустим мы ещё хотим сделать улёт фотонов из нашей системы. Введём оператор декогеренции, равный a . Его реализация, а также его добавление при генерации гамильтониана, выглядит так:

Моделирование проводится по квантовому основному уравнению. Результат представлен на Рис. ??.

В. Алгоритм Гровера

Опять же, начнём с реализация состояния. У нас будут числа с n битовым представлением. Нам также потребуется и метод для восстановления нашего числа обратно в десятичное представление. Математическую основу алгоритма Гровера можно прочесть в [?]. Реализуем сначала наше состояние.

Теперь реализуемы операторы для квантового орacula.

Построим график вероятностей для 3 битового числа (для большего числа график будет загромождён):

С. Квантовое туннелирование

Данный эксперимент подразумевает непрерывное понятие состояние. Математическая основа выложена в приложении А. Ступая на путь непрерывных задач, мы неизбежно приходим к дискретизации. Эффект квантовое туннелирования демонстрируется на базисе координат, соответственно элементом базиса у нас будет отрезок по координате. Подробнее о дискретизации непрерывных волновых функций можно посмотреть, например, в книге [?].

Для демонстрации квантового туннелирования достаточно реализовать только оператор потенциальной энергии в качестве гамильтониана. Для более сложных гамильтонианов требуется квантовое преобразование фурье (QFT), которое в библиотеке будет реализовано в будущем вместе со специальными классами, в которых задачи на базис координаты и импульс будут решаться вразы проще.

Мы должны реализовать

Демонстрация результата более наглядна на gif изображении, поэтому на Рис. ?? приведён компромисс с совмещением графиков энергии и матожидания координаты частицы.

D. BB64

Пример данного алгоритма слишком большой для данной статьи, но его упоминание примечательно. Он приведён со всеми комментариями отчётливо в примерах в репозитории библиотеки [?] (BB64_example.cpp).

IV. Производительность и параллелизация

Вся библиотека на данный момент полностью написана на пакете с открытым исходным кодом - Intel OneAPI. Он даёт серьёзное преимущество как в решении математических задач разного рода, так и в эффективности их решения, в том числе и в параллельном. Так как документация к этому пакету неточна, места написана с ошибками, а также пакет сам по себе загромождён, в QComputations присутствуют файлы `mpi_functions.(hpp,cpp)`, которые являются по своей сути интерфейсом к этому пакету. Данная информация для тех пользователей, которые хотят разобраться, как работает библиотека, для обычного пользователя данная информация излишня, поэтому мы сразу перейдём к тому, как перейти к параллельной реализации QComputations, также продемонстрируем ускорение с помощью увеличение ядер числа ядер без нитей (так как по умолчанию на каждом ядре включены нити, что является более эффективным решением в силу большого числа коммуникационных расходов, поэтому параллельная версия по большей части для суперкомпьютеров), а потом на 1 ядре от числа нитей.

При реализации библиотеки придерживались принципа, что пользователь не должен вникать в детали реализации, а также легко переключаться между многоядерной версией и одноядерной. Возьмём пример с Тависом-Каммингсом. Версия многоядерная и одноядерная для этого примера будет отличаться лишь следующими строками:

Далее код не меняется в принципе. `init_grid(ctxt)` - инициализация контекста BLACS (сетки ядер процессора), это изобретение Intel ScaLapack. Подробнее про эту сетку можно прочесть в инструкции в репозитории библиотеки [?].

Нам этом же примере продемонстрируем ускорение сначала в зависимости от числа ядер с 1 нитью на каждом (Рис. ??) и в зависимости от нитей на 1 ядре (Рис. ??)

V. Сравнение с QUTIP

Так или иначе в предыдущих разделах мы уже затрагивали минусы библиотеки Qutip. В этом разделе будет просто более обширное резюме по главным из них.

- QuantumToolBox, который Qutip и призвался осовременнить, предназначался для задач квантовой оптики, но несмотря на это, он также нашёл себе применение и в других областях ещё в 90-ые годы. Тогда им удалось красиво структурировать математические основы квантовой механики в Matlab, что и перенёс Qutip в полной мере. Но вместе с математической элегантностью пришла и её строгость, то есть все операторы, все состояния, все базисы необходи-

мо привести к строгому матричному виду, что для многих задач попросту неудобно, так как этот процесс задания матриц напрямую зависит от входных параметров. Например в Тависе-Каммингсе размер базиса, а значит и размер состояний и матричных операторов, напрямую зависят от начального состояния, например от числа фотонов, в то время как в QComputations об этом думать не нужно. Дополнительно к этому к процессу инициализации операторов и состояний в Qutip добавляются огромные требования к памяти, к вычислительным мощностям, так как инициализация требует большого количества суммирования матриц, тензорных и обычных произведений и так далее. Если в QComputations программист волен выбирать любую стратегию задания матрицы, вплоть до математической, то у Qutip есть только последняя, к тому же без блочно-распределённых матриц, в отличие от QComputations.

- Инструментарий Qutip за 13 лет разработки набрался довольно существенный, и в принципе можно было бы закрыть даже на первый минус глаза, если бы не 2 "но".

Параллелизация Qutip есть только на нитях, хотя их статья 2011 года [?] заверяет, что есть многоядерная версия, согласно одному из графиков. Только к ней не прикреплен, как ко всем остальным примерам, исходный код, а также этому противоречит огромное количество форумов и обсуждений в интернете с вопросами "Как рас-

параллелить Qutip?" или "Где скачать (или когда будет) НРС-Qutip?". На данный момент параллелизация на MPI у Qutip сделана лишь на уровне независимых друг от друга вычислений. В вопросе квантовых вычислений такие задачи являются по большей части исключением и не имеют широко характера, но даже так можно было бы сказать, что для вот конкретного класса задач Qutip хорошая библиотека, которой можно пользоваться в том числе и на суперкомпьютерах, если бы не второе "но".

При сравнении эффективности Qutip с QComputations использовались многие методы, в основном 2 главных - шрёдингер и, более показательный, КОУ. И если первый (с математической точки зрения более лёгкий) держится вдвое хуже, что для Python приемлемо, то вот для КОУ нет никаких оправданий. Замеры проводились по решению модели Тависа-Каммингса с учётом фотонов на 1 ядре с нитями (так как это единственная параллелизация, которую поддерживает Qutip). Код на QComputations представлен в примере в репозитории (realization_tc_example.cpp), код на Qutip в (Б.?). График представлен до размера гамильтониана 512. Причины этого можно посмотреть в исходном коде Qutip, а точнее, в реализации метода Адамса, на котором производятся расчёты в Qutip для КОУ.